

BoxBreaker — Informe de estado

Proyecto rpg-bbk-compiler | Estado de tareas por módulo | 20 de junio de 2026

Resumen

El proyecto traduce RPG (legacy) a BBK (lenguaje propio moderno) y, a futuro, a ejecutable. El **plugin de IntelliJ para BBK** está completo (12/12 funcionalidades, 80 tests). El **frontend RPG→BBK** traduce programas free-form completos (loop validado con el plugin) y tiene un **debugger** que muestra la traducción línea a línea. El **núcleo** (bbk-core, Java in-JVM) ya **compila y ejecuta BBK en la JVM** (lexer + AST + parser + backend a bytecode con ASM) y **ambos backends (JVM y C) cubren todo el lenguaje no-SO**: procedimientos, decimales, arrays, estructuras, subrutinas, monitor y builtins. El **runtime** (bbk-runtime) dejó de ser un scaffold: ya **persiste jobs** con ciclo de vida real (un trabajo arranca, corre en su hilo y termina), tiene **login JWT + autoridades especiales estilo IBM i** y expone una **API REST** (sign-on, usuarios y jobs — listado WRKACTJOB-style + envío de trabajo) sobre MySQL/JPA con logging a archivo. Encima se construyó un **visor .NET (RuntimeVisor)** estilo pantalla verde 5250 que consume esa API (sign-on → menú → trabajos activos), con **i18n ES/EN**. Es justo la pieza que no existe en open source y la base sobre la que crecería una versión profesional.

plugin-bbk COMPLETO 12/12 features · 80 tests	rpg-frontend + debugger TRADUCE + VALIDADO free-form · 63 tests	bbk-core (núcleo) EJECUTA BBK (2 backends) bytecode+C · 97 tests	bbk-runtime JOBS + AUTH + API JWT · MySQL · 3 tests	RuntimeVisor (.NET) VISOR 5250 WRKACTJOB · i18n ES/EN
--	--	---	--	--

1 plugin-bbk — Soporte de lenguaje BBK en IntelliJ

12 funcionalidades de asistencia de editor. Estado: **completo**, 80 tests automatizados verdes.

Funcionalidad	Estado	Detalle
1. Autocomplete básico	✓ Hecho	Ctrl+Space: keywords, tipos, modificadores según contexto
2. Autocomplete de identificadores	✓ Hecho	Variables, procs, constantes declaradas en scope
3. Autocomplete de miembros	✓ Hecho	employee.<campo> sugiere subfields del DS (incl. cross-file)
4. Live templates	✓ Hecho	dcls + Tab expande a DCL-S; 3 contextos
5. Go to declaration	✓ Hecho	Ctrl+B / Ctrl+Click salta a la declaración
6. Find usages	✓ Hecho	Alt+F7, intra-archivo y cross-file vía stub index
7. Rename refactor	✓ Hecho	Shift+F6 renombra en todos los usos; valida reservadas
8. Búsqueda de símbolos cross-file	✓ Hecho	Ctrl+Alt+Shift+N por nombre en todo el proyecto
9. Smart completion	✓ Hecho	Ctrl+Shift+Space filtra candidatos por tipo esperado
10. Inspections con tipos	✓ Hecho	7 inspecciones: mismatch en asignación/return/args/condición/INZ + refs sin resolver
11. Parameter info	✓ Hecho	Ctrl+P muestra la firma con el parámetro actual en negrita

Funcionalidad	Estado	Detalle
12. Quick documentation	✓ Hecho	Ctrl+Q para declaraciones y 21 builtins catalogados

Infraestructura de soporte (toda implementada)

Componente	Estado	Detalle
Lexer + parser (BBK.bnf)	✓ Hecho	JFlex + Grammar-Kit: todas las construcciones del lenguaje
PSI + mixins + factory	✓ Hecho	Naming, setName, getTextOffset, síntesis de IDENT
Scope (3 impls + walker)	✓ Hecho	Module / Procedure / Block scope
References (4 + contributor)	✓ Hecho	Ident / Member / Subroutine / Type
Stubs + indexes (7+7)	✓ Hecho	Resolución cross-file persistente
Type system (types/)	✓ Hecho	BbkType, inferencia, assignability con promociones
Editor polish	✓ Hecho	Brace matcher, smart typing, prefix matcher con hyphen

Bug resuelto recientemente: falso positivo «Unresolved reference» en member-access cross-file (currentOrder.orderId). Causa: BbkTypeResolver no seguía el LIKEDS hacia un DS de otro archivo. Fix: fallback al índice de proyecto. Beneficio extra: ahora el member-access cross-file resuelve de verdad (Ctrl+B y autocomplete de miembros).

2 rpg-frontend — Traductor RPG → BBK

Recién iniciado. Decisiones tomadas: entrada **free-format RPGLE**, lenguaje **Java**, salida **texto .bbk** (validable con el plugin). Pipeline objetivo: *RPG source* → *lexer* → *parser* → *AST* → *emitter* → *.bbk*.

Tarea	Estado	Detalle
Setup del módulo	✓ Hecho	rpg-frontend en Gradle (Java 21 + JUnit), registrado en settings
RpgLexer + tokens (free-form completo)	✓ Hecho	Toda la superficie léxica fully-free (grammar §2): IDENT, keywords, %BIFs, 9 literales, operadores, `` calificado, directivas /COPY //IF. 19 tests
AST de RPG (completo, §4.2/4.3/4.4)	✓ Hecho	sealed interfaces + records: 10 declaraciones, 15 statements, 9 expresiones, tipos (Scalar/Like/LikeDs/LikeRec). 5 tests
RpgParser (gramática free-form completa)	✓ Hecho	Recursive descent: todas las declaraciones (ctl-opt, dcl-s/c/ds/pr/pi/f/proc) y statements (if, select, dow, dou, for, monitor, begsr, file ops...); precedencia §4.4; resuelve la ambigüedad del '='. 20 tests
BbkEmitter (RPG → texto BBK)	✓ Hecho	Traduce toda la superficie: tipos en mayús, '='→'==', loops → llaves (while/do-while/for), dcl-pi → parámetros inline, ':'→',', figurativas. 19 tests
Validación del loop con plugin-bbk	✓ Hecho	8 tests round-trip (RPG → BBK → parser del plugin, 0 errores). Encontró y corrigió un mismatch: %subst → substr (BBK no usa el prefijo '%')

63 tests verdes en rpg-frontend. El frontend **traduce programas RPG free-form completos a BBK** (todas las declaraciones, statements y expresiones de la gramática §4) y el **loop está verificado**: el BBK generado parsea sin errores con plugin-bbk (test round-trip en el build del plugin). El '*' se desambigua por contexto. Único gap léxico diferido: asignación compuesta (`+= -= \*= /=`), ausente en la gramática. Detalle en

[docs/rpg-frontend/overview.md](#).

Módulo debugger: muestra la traducción línea a línea (RPG a la izquierda con número de línea, BBK a la derecha) más el BBK completo. Ejecutable: `gradlew :debugger:run --args="examples/sample.rpgle"`. 5 tests.

3 Núcleo del compilador (bbk-core) — ejecuta BBK

Toma el texto BBK del frontend y lo compila. **Java in-JVM** (el plugin lo invoca en el mismo proceso, sin binarios nativos). Un solo AST de BBK alimenta **dos backends**: BBK → **bytecode JVM** (corre in-process) y BBK → **C** (compila con gcc), ambos a la par. No hay IR separado: el AST + su análisis es el IR (evita duplicar). Parser headless propio (el del plugin está atado a IntelliJ).

Tarea	Estado	Detalle
Lenguaje del núcleo	✓ Hecho	Java in-JVM (decidido)
Lexer de BBK	✓ Hecho	Completo, grounded en BBK.bnf: case-sensitive, operadores C-style, hex/oct/float/dec
AST de BBK	✓ Hecho	Completo: 9 declaraciones, 17 statements, 11 expresiones (ternario, bitwise, [] vs ())
Parser de BBK	✓ Hecho	Toda la gramática; sin ambigüedad del '=' (== vs =); parsea el output del frontend
Backend BBK → bytecode JVM	✓ Hecho	ASM → clase bbk.Main; corre in-process en la JVM. Enteros/bool/strings, aritm./bitwise/lógica/ternario, if/while/do-while/for/select, break/continue, print. <code>gradlew :bbk-core:run</code>
Backend JVM: lenguaje no-SO completo	✓ Hecho	Procedimientos (DCL-PROC→métodos, recursión, CTL-OPT MAIN), decimales exactos (PACKED/ZONED→BigDecimal con escala+redondeo @halfup/@trunc), FLOAT, arrays (incl. arrays de DS), estructuras (DCL-DS/TEMPLATE/LIKEDS), subrutinas (BEGSR/EXSR/LEAVESR), monitor/on-error, constantes, valores especiales, 15 builtins puros. 73 tests. Deferido: OVERLAY, fechas, file/EXTPGM (SO)
Backend BBK → C: lenguaje no-SO completo	✓ Hecho	A la par del JVM: emite C self-contained (prelude de helpers string/decimal/monitor). Procedimientos, decimales (long double con escala), arrays (incl. de DS), estructuras, subrutinas, monitor (setjmp), builtins. Verificado con gcc real (WinLibs MinGW 16.1): compila+corre vía <code>--run-c</code> , los tests gated ejecutan (no se saltan). Deferido: OVERLAY, fechas, file (SO)
Análisis semántico compartido	✓ Hecho	Paquete semantic: un solo analizador resuelve nombres + infiere el tipo de cada expresión (Type neutral) y junta diagnósticos. Los dos backends consumen el SemanticModel (borrado el typeOf/lookup duplicado); solo conservan el storage físico. Unificó ** a FLOAT
docs/bbk-spec.md	✓ Hecho	Spec formal: léxico, sistema de tipos + torre numérica, declaraciones, sentencias, precedencia de operadores, builtins, valores especiales, lowering (qué entra y qué no) y EBNF. Grounded en bbk-core
Lenguaje de bbk-runtime	✓ Hecho	Spring Boot, servicio REST standalone (decidido sobre Rust)

97 tests verdes en bbk-core. El loop completo del proyecto cierra de punta a punta: RPG → frontend → texto BBK → bbk-core (parser → AST) → **dos backends** (bytecode JVM que corre in-process, y C que **compila+corre con gcc real** — WinLibs MinGW 16.1). Verificado end-to-end en ambos: `factorial(5)=120`, `price(199.95)*qty(3)=599.85` (escala 2), arrays de DS, procedimientos, select, monitor atrapando división por cero. La verificación con gcc destapó y corrigió dos bugs de MinGW + long double (stdio ANSI para %Lf; snap del truncado decimal). Decisiones abiertas: BigDecimal (JVM) / long double (C) vs BCD propio exacto; arrays 0-based.

4 Runtime (bbk-runtime) — jobs + auth + API REST

Servicio **Spring Boot standalone** invocado por **REST/HTTP** (elegido sobre Rust). Maneja la superficie *gruesa* de IBM i; la aritmética BCD/strings **no** cruza la red — queda local en bbk-core / el código generado (decisión #4 intacta). Ya dejó de ser scaffold: **persiste jobs con ciclo de vida real**, tiene **login JWT con autoridades especiales** estilo IBM i y una **API REST** operativa sobre MySQL/JPA.

Tarea	Estado	Detalle
Base Spring Boot	✓ Hecho	Spring Boot 3.5.5, Gradle Kotlin DSL, Java 21 (release 21 sobre JDK 25). Persistencia MySQL + JPA/Hibernate (ddl-auto update) y logging a archivo (Logback con rolling)
Jobs persistidos + ciclo de vida	✓ Hecho	Entidad Job (número estilo IBM i, estado ACTIVE/ENDED) con historial de eventos (audit BPMS-style) y atributos EAV . Un trabajo arranca, corre en su propio hilo y termina (JobRunner sincrónico y asíncrono)
Library list (*LIBL)	✓ Hecho	JobLibrary ordenada: path de búsqueda de objetos por job
Login JWT + autoridades especiales	✓ Hecho	UserProfile con hash BCrypt y special authorities (*ALLOBJ, *SECADM, *JOBCTL...). Sign-on emite JWT (jwt); Spring Security stateless; perfil semilla QSECOFR
API REST	✓ Hecho	/api/auth (login + me), /api/users (alta/listado, exige *SECADM) y /api/jobs: listado WRKACTJOB-style + POST de un trabajo que espera (exige *JOBCTL). Autorización por endpoint (401/403)
Acceso a datos / registros (DDS)	■ Pendiente	Capa record-level sobre las tablas de negocio
Program calls (CALL a *PGM)	■ Pendiente	Invocación de otros programas dentro de un job
Activation groups / Spool files	■ Pendiente	Grupos de activación y archivos de spool
Lado cliente (bytecode / C-AOT → runtime)	■ Pendiente	Que el código generado arranque un job y llame al runtime por HTTP

3 tests en bbk-runtime sobre **MySQL real**: ciclo de vida de jobs, sign-on/JWT y contextLoads. Verificado end-to-end: login QSECOFR → GET /api/jobs → grid del visor; un trabajo BBKWAIT enviado por POST aparece **ACTIVE** y pasa solo a **ENDED** al terminar su espera.

5 RuntimeVisor — visor .NET (pantalla verde 5250)

Cliente de escritorio **.NET WinForms (.NET Framework 4.8)** que consume la API del runtime con estética de terminal **5250** (verde sobre negro, Consolas). Practica .NET y, a la vez, le pone cara al runtime. Flujo: *sign-on* → *Home* (cabecera 5250) → *trabajos activos*, navegación híbrida (comando WRKACTJOB / opción numérica + teclas de función).

Tarea	Estado	Detalle
Sign-on contra el runtime	✓ Hecho	Login JWT (usuario/contraseña → token guardado en la sesión); mensajes de error claros
Home estilo 5250	✓ Hecho	Cabecera Sistema/Subsistema/Terminal/Usuario/Fecha/Hora con reloj en vivo; se colapsa al navegar
WRKACTJOB (trabajos activos)	✓ Hecho	Grilla que consulta GET /api/jobs y muestra número/usuario/nombre/estado/creado/terminado. F5 refresca, F3 vuelve
Internacionalización (i18n)	✓ Hecho	.resx + ResourceManager: español (base) e inglés (satélite). Idioma por cultura del SO u override - - lang=en / BBK_LANG
Enviar trabajo desde el visor	■ Pendiente	Un F6 que haga el POST y refresque, para lanzar y ver un job activo sin salir
Vistas adicionales (usuarios, etc.)	■ Pendiente	Reusar la navegación para más superficies del runtime

Compila con MSBuild de Visual Studio. Pendiente de higiene: quitar las credenciales precargadas (QSECOFR/qsecofr) y agregar .gitignore para los artefactos .NET (.vs/, obj/, bin/).

6 Otros módulos — sin empezar

Tarea	Estado	Detalle
plugin-rpg: highlighting	■ Pendiente	Resaltado de sintaxis RPG en IntelliJ
plugin-rpg: editor parser	■ Pendiente	Errores inline en RPG
plugin-rpg: 'translate to BBK'	■ Pendiente	Comando que invoca el rpg-frontend
boxbreaker-ide: bundle	■ Pendiente	IDE empaquetada sobre IntelliJ Platform SDK (solo scaffolding)
boxbreaker-ide: branding + installer	■ Pendiente	Splash, icono, build de distribución
Testing E2E	■ Pendiente	Compilar RPG → ejecutar → verificar; equivalencia intérprete vs AOT
Distribución	■ Pendiente	Publicar plugins en Marketplace; distribuir la IDE

7 Deuda técnica / higiene pendiente

Tarea	Estado	Detalle
Credenciales precargadas en el visor	■ Pendiente	Form1 trae QSECOFR/qsecofr hardcoded para no tipear; quitar antes de uso real (ver TODO.md del visor)
.gitignore para el módulo .NET	■ Pendiente	Se cuelan como untracked .vs/, obj/, bin/, *.suo, *.vsidx
Secreto JWT y datasource de dev	■ Pendiente	application.properties tiene secreto JWT y root/root de MySQL en claro (solo dev)
Quitar logs de diagnóstico	■ Pendiente	BBK-REF / BBK-RESOLVE / BBK-SCOPE quedaron del debug de cross-file

Tarea	Estado	Detalle
Unificar catálogo de builtins	■ Pendiente	BbkBifProvider tiene lista hardcodeada en paralelo al nuevo registry
Inspections V1.5 + quick-fixes	■ Pendiente	Unused / Shadowed / ReservedWord / LikeCycle + fixes (diferidas)

Generado automáticamente a partir del estado del repositorio. Leyenda: ✓ **Hecho** ● **En curso** ■ **Pendiente**.